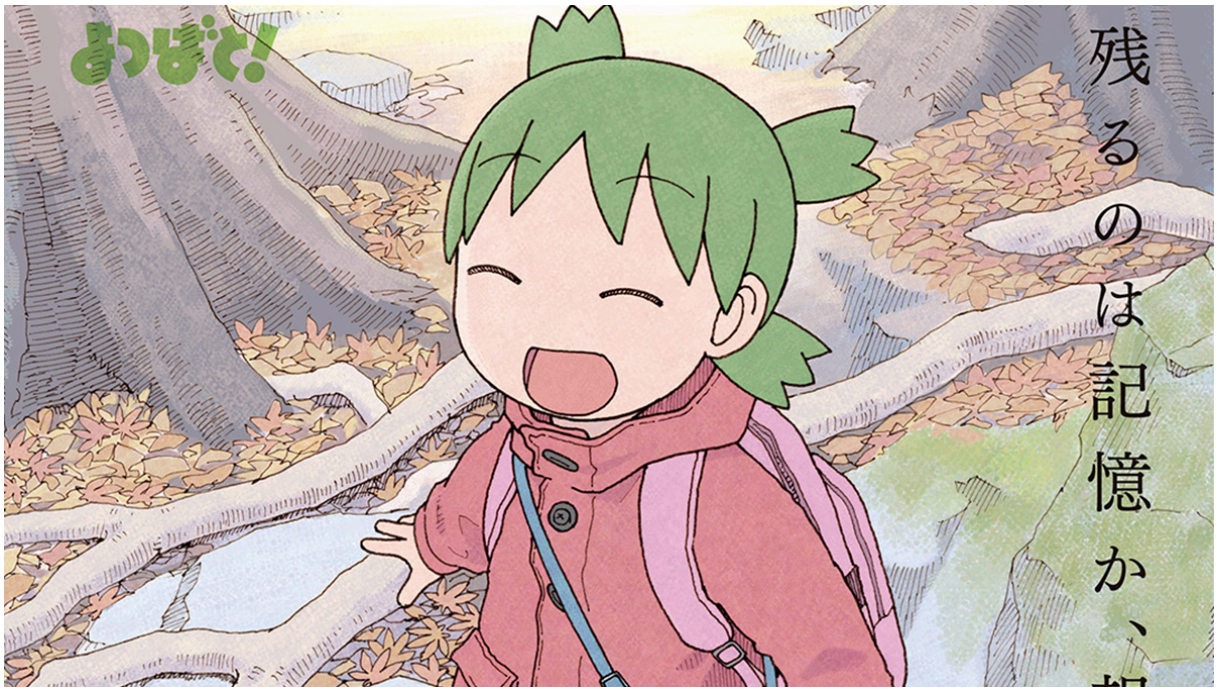
CMSC 216 Notes

Introduction to Computer Systems

Miagao



Yotsuba from Yotsuba&!

Contents

Course Intro 3
- Linux Shell Filepaths 3
- Linux Shell Commands 3
- Valgrind 4
- emacs 4

C Language I 5
- C versus Java 5
- C Program Example 5
- Compiling a C program 5
- Compilation Stages 6
- Types in C 6
- Numerical Literals 7
- Variable Declaration 7
- Arithmetic Operators 7
- Boolean Type and Operators 8
- Precedence and Associativity 8
- Assignment Statements 8
- Output using `printf()` 9
- Input using `scanf()` 9
- C Control Statements 10
- Additional Formatting Options 10
- C Functions 10

C Pointers 12
- The `sizeof` operator 12
- Pointers 12
- Dereferencing a Pointer 13
- Pointers as Parameters 13
- Garbage Pointers 14
- Pointers to Local Variables 14
- `NULL` Pointer 14
- The `const` modifier 14

Arrays 16
- Arrays in C (static) 16
- Array Initialization 16
- Symbolic Constants 16
- Array Parameters 17

Course Intro

C language is a high-level language close to assembly.
- Exposes how data is organized in a linear address space.
- **Pointers** demonstrate how variables represent locations in memory.
- Shows system calls provided by the OS.

Assembly language shows what hardware can do:
- Instructions processors can execute.
- How function calls work.
- How input/output devices are accessed.

UNIX operating system
- Shows how threads, processes, pipes, etc are building blocks for servers and high-performance applications.
- We will use a variant of UNIX called **Linux**.

Linux Shell Filepaths

Directories in filepaths are separated by forward slashes (/). Specific characters denote specific directories.
- The slash (/) refers to the root directory.
- The dot (.) refers to the current directory.
- The two dots (..) refer to the parent directory.
- The tilde (~) refers to the home directory.

Linux Shell Commands

Flags are extra options for commands. They are generally denoted by a dash.

The following commands are the *essentials* for writing and executing C programs.

1) `pwd` displays the current direction. Forward slashes separate directories and files.
2) `ls` displays all files and directories in the current directory.
   a) `ls -al` will also include hidden directories and files.
   b) `ls -F` will append specific characters to items' names depending. Particularly, directories are marked with a forward slash.
   c) `ls *.EXTENSION` displays all files with the provided EXTENSION.
3) `cd` changes the directory to the home directory.
   a) `cd DIRECTORY_NAME` changes the directory to the specified directory.
   b) `cd ~` makes the current directory the home directory.
4) `mkdir` creates a new directory in the current directory.
5) `cp FILE_1 FILE_2` will create a new file using the data of the first file.
   a) `cp -r FILE_1 FILE_2` will create a new file using the data of the first file, but recursively. **This is required when copying folders.**
6) `rm FILE` permanently removes the specified file.
7) `mv` moves files/directories to a directory or renames files/directories.

   a) `mv FILE_1 FILE_2` renames the first file to `FILE_2`.
   b) `mv FILE DIRECTORY` moves the file into the specified directory.
   c) `mv DIRECTORY_1 DIRECTORY_2` renames the first directory to `DIRECTORY_2` if the second directory doesn't exist. Otherwise, the first directory is moved to the second directory.
8) `rm DIRECTORY` removes the specified *empty* directory.
   a) `rm -f -r DIRECTORY` removes the specified directory, *even if it is non-empty.*
9) `man COMMAND` pulls up the documentation for a given command.
   a) `man SECTION_NUMBER COMMAND` pulls up the specified section for the documentation of the given command.
10) `diff FILE_1 FILE_2` shows the differences between `FILE_1` and `FILE_2`.
   a) `diff -bwi FILE_1 FILE_2` also ignores whitespace differences.
   b) `diff -Bwi FILE_1 FILE_2` also ignores whitespace and newline differences.

Valgrind

The **valgrind** utility is a tool that provides extra information when compiling a program.

- valgrind can help locate problems in code when there is a segfault.
- valgrind can also detect invalid memory operations.
- To use the valgrind utility, compile your program with the `-g` flag, then execute using valgrind.
  - `valgrind a.out`
  - `valgrind --track-origins=yes a.out`
    - This shows were uninitialized values come from.
  - `valgrind a.out < public01.in`
    - Using valgrind with an input file and input redirection.
  - `valgrind a.out < data.txt > results`
    - Using valgrind with input and output redirection.

emacs

The following shortcuts are important.

- ^X + ^C quits.
- ^X + ^S saves a file.
- ^X + ^F opens a file.
- ^K kills the process.
- ^Y yanks.
- ^S forward searchs.
- ^R reverse searchs.
- ^_ undos last change.

C Language I

For this course, we will be using C99.

- C is not object oriented, so instead of the term *method*, we use **function**.
- To stop a C program, use ctrl + c.

C versus Java

- C is more performant because it trusts the programmer to manage details such as array bounds, memory leaks, and error checking.
  - Java's garbage collection and constraints hurt performance.
- C is the next level after assembly, thus it's used for writing operating systems such as Unix.
- C compiles to machine code which the processor can directly execute. Java requires an intermediate step of converting to bytecode.
- As mentioned prior, C lacks explicit object-oriented features.

C Program Example

```c
#include <studio.h>
int main() {
  printf("Fear the turtle\n");
  return 0;
}
```

- `#include` allows the compiler to include other files (generally .h **header files**) for their function prototypes and type/variable declarations.
  - Standard system files are delimited by **<>** while user-written files are delimited by **“”**.
- The `main()` function is where all C programs begin execution.
  - Return values from this function indicate the end of a program.
  - By convention, an exit code of 0 means that everything went well.

Compiling a C program

C programs must be compiled to be executed.

- The **gcc** program is will be used to build our programs.
  - `gcc SOURCE_FILES`
- The common flags for gcc are as follows:
  - `-g` enables debugging.
  - `-Wall` warns about potential problems.
  - `-o FILENAME` places the executable in the specified file.
  - `-c` compiles to object file (without link).
  - `-lm` compiles with the C math module (if imported).

The following are examples of compiling single files and multiple files at once.

```
1   gcc file.c                                                    UNIX
2   gcc -g -Wall -o prog1 prog1.c tools.c
```

To run executables in UNIX, type the name of the executable. In some cases, the executable's name should be preceded by "./".

Compilation Stages

- **Preprocessor**
  - *Does text-processing of source code.*
  - Used to provide declarations shared by different source code.
  - Directives begin with the pound sign, #, but do not end with a semicolon.
- **Translation**
  - *An object file **(.o)** is created.*
  - Ensures individual files are consistent within themselves.
- **Linking**
  - *Joins one or more compiled object files together.*
  - *Creates an executable file.* Note that by UNIX convention, executable files lack a filename extension

Types in C

- char, short, int, long int.
  - These can be signed or unsigned.
- float, double
- * is a pointer to a variable.
- [] represents arrays of variables.
- union {} represents variables sharing the same space.
- struct {} represents the concatenation of variables.
- enum {} represents named integer values.

Type name	Minimum size (bytes)	Size on *grace* (bytes)
char	1	1
short	2	2
int	2	4
long	4	8
float	4	4
double	8	8
long double	10	16

Numerical Literals

Suffixes can be used to specify a numerical literal's type.
- "L" can be used to specify "long."
- "f" can be used to specify "float."
  - By default, a decimal is a double.

Prefixes can be used to specify bases other than base 10.
- "0" represents octal.
- "0x" represents hexadecimal.

Variable Declaration

Variables may be declared as follows.

typename var1, var2, ...;

Variables may also be initialized during declaration.

```c
int x = 6;
```

Even if an initial value is not given, declaration always reserves space for the variable in memory. In the example above, 4 bytes will be reserved even if the initial value of 6 was not given.
- However, it is recommended to give an initial value, as C will not stop you from using an uninitialized variable. In fact, it will use whatever data is in the reserved memory space.

Arithmetic Operators

The basic arithmetic operators in C are analogous to Java.
- + add
- - subtract
- / divide
- % remainder
- * multiplication

Compound assignment operators are basic arithmetic operators followed by an equals sign.
- +=, -=, /=, %=, *=
- ++ increment
- -- decrement
- **(typename)** casting

> **Note**
>
> Integer division occurs only if both operands are integers. Otherwise, floating point division occurs.

Boolean Type and Operators

There is no boolean type in C. Integers are used instead.
- 0 represents **false**.
- Any other integer represents **true**. By convention, this is 1.

Similar to arithmetic operators, the boolean operators are analogous to Java.
- Relational operators: <, >, <=, >=
- Equality operators: ==, !=
- Logical operators: &&, ||, !

Because integers are used instead of a true boolean type, these either evaluate to **0 (false)** or **1 (true)**

C
```c
int i;
i = (!3) == (4 < 2);
i = (!2) || (5 && i);
```

Java
```java
boolean i;
i = (!true) == (4 < 2);
i = (!true) || (5 && i);
```

Precedence and Associativity

some of these fellas go first. Lowk PEMDAS.

Assignment Statements

The **assignment operator** in C is the equals symbol, =.
- It is right associative. A value is returned from the operator to the variable on the left side of the assignment operator.
- *Note, this means that variable assignment is an expression. Consequently, you can write any expressions as individual statements.*

Generally, expression statements are most useful with the assignment operators.

C
```c
int x, y;
y = x = 123; //x is assigned to 123, then y is assigned to 123 transitively through x.
y = 5 + (x = 3); //x is assigned to 3 which yields 3, then y is assigned to 5 + 3 = 8.
```

Output using `printf()`

The `printf()` function is formally declared in stdio.h. It can print strings, which may be formatted with a variable number of expression statements.

printf("formatstring", expression1, expression2, expression3, …**)**;

The expressions are inserted into the format string using format specifiers. These correspond to each passed-in expression, and decide how each value will be interpreted.

- %d prints an integer in decimal.
- %ld prints a long integer.
- %u prints an unsigned integer.
- %f prints a floating point value.
- %c prints a character.
- %s prints a character string.
- %x prints in hexadecimal.
  - %X makes the letters capital.
- %e prints in exponential form (e notation).
- %% prints the percent symbol.

Like Java, **escape sequences** can be used in strings to specify things that would be otherwise inconvenient to type out.

- \n creates a new line.
- \t creates a horizontal tab.

Input using `scanf()`

The `scanf()` function is formally decalred in stdio.h.

scanf("formatstring", **&**variable1, **&**variable2, **&**variable3, …**)**;

The format string specifies the inputs being read using format specifiers and inserts them into each passed-in variable.

- Format specifiers should match the type of their corresponding variables. If this is not the case, the function will cease.
  - *Most* specifiers skip leading whitespace.
- Whitespace in the format string simply skips any whitespace input at that point.
- Other characters just tell the function to read the corresponding characters in the input stream.

The `scanf()` function **returns the number of items assigned to variables or the EOF (End of File Marker).**

> **Note**
>
> The ampersand symbols associated with each variable means that the address of each variable is being retrieved.

> **Caution**
>
> Avoid adding whitespace at the end of the format string of a `scanf()` function.

C Control Statements

Just like Java, C has **if**/**else**/**else if**, **for**, **while**, and **switch** statements.

The flow of loops can be controlled using the following keywords:
- **break** immediately ends a loop.
- **continue** skips the remainder of the loop and advances to the next iteration.

Compound statements are delimited by {}, and can go anywhere a single statement can. Variables defined within are limited to the scope of the compound statement.

Additional Formatting Options

We can specify field width, precision, etc alongside format specifiers.
- **-** left-aligns the output within the field width.
- **0** pads the output with leading zeros when possible.
- **+** displays a plus sign for positive signed numbers.
- An integer specifies the width (minimum characters).
- An period followed by an integer specifies the precision.
  - Floating-point: number of digits
  - Integer: minimum digits
  - String: maximum characters

C Functions

Functions in C follow the following syntax:

---

returnType functionName(parameters) {/* statements */}

They are called as follows:

---

functionName(arguments);

Function prototypes provide information about a function's return type and parameter types, but do not define it.
- The compiler can use them to verify the correctness of function calls.

- Thus, they allow functions to be defined after the main function in a C program.

---

returnType functionName(parameterTypes);

> **Important**
>
> Unlike Java, functions **cannot be overloaded.**

> **Caution**
>
> Previously, variables in compound statements could only be declared at the top. In C99, mixed declarations are now allowed. This primarily affected for-loops.

C Pointers

The `sizeof` operator

The `sizeof` operator is *unary*, and evaluates to the bytes needed to store the operand.

- The operand can be either an expression or a type name.

> **Note**
>
> The expression is not evaluated.

- Return type is long.
- For example:

```c
int i = 67;
printf("%ld\n", sizeof(i)); //Prints 4 (on grace)
printf("%ld\n", sizeof(unsigned char)); //Prints 1
printf("%ld\n", sizeof(++i)); //Prints 4 (on grace)
printf("%ld\n", i); //Prints 67, the line 4 expression isn't evaluated
```

Pointers

A **pointers** is a memory address or reference to a variable. They have two attributes:

- **Address:** Address of a byte of memory.
- **Type:** Type of the data starting at the address.
- A **pointer variable** is a variable that holds a pointer value.
  - Many times, they are referred to as just *pointers*.

Variables are denoted as pointers with an asterix (*) before it:

```c
int *p; //Creates a variable ip, whose type is "pointer to int"
```

- Due to how they're denoted, the following are also equivalent:
  - `int*p`
  - `int * p`
  - `int* p`

An ampersand (&) before an entity *denotes pointer to it* (specifically, the address of its first byte).

The *address of an int variable* can be the value of an *int pointer variable.*

- We can access the entity a pointer variable points to.
- The type attribute of a pointer is used by the compiler when generating object code, but the attribute itself is not stored in object code.

```c
int i = 6;
int *p;
```

```c
p = &i;
printf("%d %d\n", *p, *(&i));
```

While addresses look like integers, they are a different type. For instance, the following does not compile:

```c
int x = 20;
int *m = x;
```

> **Note**
>
> The format specifier for addresses is `%p`, although it requires a null pointer variable for use in `printf()`. Thus, casting using `(void *)` can be used.

> **Note**
>
> The size of a pointer depends on the system. Pointers in a 32-bit system are 4 bytes and pointers in a 64-bit system are 8 bytes.

Dereferencing a Pointer

An asterix (*) before a pointer *dereferences the pointer*.

By dereferencing a pointer variable, we can access the value present at the address being pointed to.

```c
int i = 6, j, *p;
p = &i;
j = *p; //Dereferences p, j is set to 6
*p = 4; //Points to the address of 4
```

Pointers as Parameters

> **Important**
>
> Everything in C is passed by value.

Below is a good example of a function that needs pointers as parameters

```c
void swap(int *a, int *b) {
  int temp = *a;
  *a = *b;
  *b = temp;
}
```

Because things are passed by value, pointers are needed to change the values of the variables provided in the function call.

Garbage Pointers

When a pointer is defined, it points to the address associated with the memory location allocated for the pointer variable. If pointed to an address where no value has been initialized:

- A **segmentation fault** (**segfault**), accessing a memory location that you don't have permission to access.
- A **bus error**, violating the computer's memory alignment policies.
- A **silent failure**, where everything appears to run correctly, but the value being accessed may change later on.

Pointers to Local Variables

> **Caution**
>
> Do not return pointers to local variables. Once the function call has finished, the memory used by the variable is not supposed to be accessed.

NULL Pointer

A **NULL pointer** points to the address 0, where nothing is allowed to be accessed.

- This is defined in stddef.h, which is included by many header files.

```c
#define NULL ((void*)0)
```

- **NULL** is analogous to **null** in Java.
  - Dereferencing a NULL pointer usually results in a segfault, as you are generally not supposed to access the address 0.
- NULL can be assigned to any kind of pointer variable, so checking whether or not a pointer is NULL prior to dereferencing is safest.

The `const` modifier

The `const` modifier indicates that a variable cannot be changed. This is enforced by the compiler. The order of the type specifier and the modifier does not matter.

```c
int const k = 4;
const int m = 5;
k++; //Error
m++; //Error
```

The order of the type specifier and the modifier *does matter when using pointers*:

```c
int i = 4, j = 5;
const int *p = &i; //Pointer variable to constant int
int * const q = &j; //Constant pointer variable to int

p = &j; //WORKS
*p += 5; //ERROR
q = &i; //ERROR
*q += 23; //WORKS
```

- If a pointer variable points to a constant value, then it is that value that cannot be changed.
- If a constant pointer variable points to a value, then it is the pointer value that cannot be changed.

Arrays

Arrays in C (static)

Like Java arrays, all elements in an **array** are of the same type and indexed using the [] operator.

- In C, elements of an array are next to each other in memory as opposed to being an object like in Java.
- Additionally, the user has to keep track of the size, as there is no length property.
- The syntax between them is slightly different too.

```c
//array of 5 ints
int a[5];
```

```java
//array of 5 ints
int[] a = new int[5]
```

Arrays can be viewed as a pointer to the first byte of the first element of the array.

- *The name itself is not a pointer.* Rather, the compiler converts it to a pointer when needed.
- The (constant) size of an array must be known at compile time.

> **Caution**
>
> An assignment operator cannot be used with two array operands.

However, using == and != does work. However, this does *pointer comparison*.

Array Initialization

When initializing arrays, you can list the elements separated by commas and in-between braces:

```c
int arr[5] = {1, 2, 3, 4, 5};
```

- Because assignment differs to initialization, the notation above does not work if the array has been previously declared.
- 0s pad the array if the initializer is not complete.
- A size does not need to be provided for an initialized array, as the size is *implicit*.
- Initializers with excessive elements cause compiler errors.

Symbolic Constants

Symbolic constants can be created using the #define preprocessor directive:

```
#define name value
```

All occurences of *name* in the program will be replaced with *value*.

- This is useful for defining constants for important values such as array sizes to improve code maintainability.

```c
#include <stdio.h>
#define ARRAY_LENGTH 3

int main() {
  int i, a[ARRAY_LENGTH] = {10, 20, 30};
  printf("[%d", a[0]);
  for (i = 1; i < ARRAY_LENGTH; i++) {
    printf(", %d", a[i]);
  }
  printf("]\n");

  return 0;
}
```

Array Parameters

Recall that arrays are pointers to the first byte of the first element of the array.

- The parameter declaration int a[] is equivalent to int *a.
- Thus, modifications done to an array parameter are done to the passed-in array.
- Consequently, the defined array parameter size does not matter. *Only the type matters.*

> **Note**
>
> Passing in the array parameter's size as a different parameter is common practice.